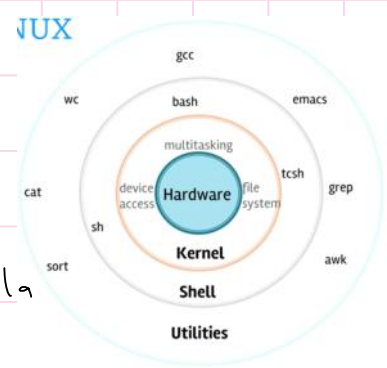


LINUX



JUX



Shell programa que interpreta las órdenes que escribimos en la terminal para interactuar con el sistema operativo.

Nos provee de comandos, estructuras de control y variables de ambiente.
Por defecto usamos BASH (bash es el lenguaje)

Sistema de archivos (File System) Jerarquía de un sistema de archivos



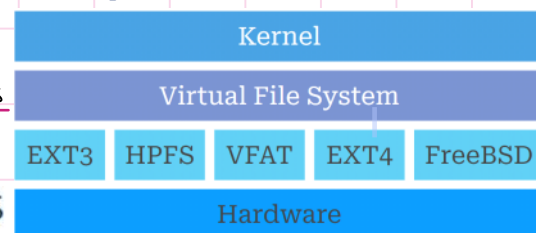
/bin/	Binarios de comandos de usuario esenciales	
/boot/	Archivos estáticos del cargador	
/dev/	Archivos del dispositivo	
/etc/	Configuración del Sistema específica del host	/ home / Desktop
/home/	Directorio de inicio de usuario	/ home / Documents
/lib/	Bibliotecas compartidas y módulos del kernel esenciales	
/media/	Punto de montaje para medios desmontables	
/mnt/	Punto de montaje para un sistema de archivos temporal	
/opt/	Paquetes de software de aplicaciones adicionales	
/sbin/	Binarios del sistema op.	
/srv/	Datos del servicio Proporcionados por el sist. op.	
/tmp/	Archivos temporales.	
/usr/	Utilidades y aplicaciones de usuario	usr /local
/var/	Archivos variables	/ usr/ local/ bi
		/ usr/ local/ games
/root/	Directorio de inicio para el usuario raíz	
/proc/	Núcleo de documentación del sistema de archivos virtual y estado de procesos	

Implementación del sistema de archivos de dos partes

línea de comandos

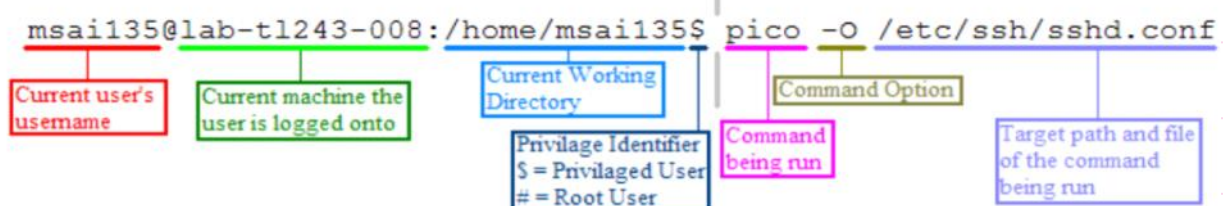
Prompt:

<username>@<computer>:<path>\$



msai135@lab-tl243-008:/home/msai135\$

Command Prompt with information <<<<>>>> Command entry



Navegar en el File System

- ⊛ Pwd Nos indica el directorio actual
- ⊛ ls lista de los archivos de este directorio
- ⊛ cd cambio de directorio



cd

cambio de directorio

Caracteres especiales

- ~ tu directorio home
- directorio actual
- .. directorio padre
- * comodín (nombre)
- ? comodín (caracter)
- TAB Intenta completar cualquier nombre parcialmente escrito.

Comando <ls>

- ls - a → lista todos los archivos (incluido ocultos ".")
- ls - l → lista los detalles de un directorio
- ls - lr → " en forma recursiva
- ls - lh → lista el directorio en detalle
- ls - ls → lista ordenado el directorio por tamaño
- ls - lt → lista los archivos ordenados por fecha de creación.

head -n N [file] - Display first N lines

tail -n N [file] - Display last N lines

ln -s [file] [new] - Create symbolic link

cat [file] [file2...] - Display file(s)

tac [file] [file2...] - Display file in reverse order

touch [file] - Update modification time

od [file] - Display file contents, esp. binary

Comandos muy usados

cp [file1] [file2] - Copy file

mkdir [name] - Make directory

rmdir [name] - Remove (empty) directory

mv [file] [destination] - Move/rename file

rm [file] - Remove (-r for recursive)

file [file] - Identify file type

less [file] - Page through file

¿qué hace un comando? help, man, info

⊛ Recorrer el sistema de archivos (<cd>)

⊛ Ver el contenido, seleccionar y procesar archivos de texto (<cat>, <more>, <less>),

⊛ Buscar adentro de uno o más archivos algún dato (<grep>) (<ls>, <mkdir>)

⊛ Crear y borrar archivos (<touch> y <rm>) ...

Procesamiento de texto

awk - Pattern scanning and processing

language

cat - Display file(s)

cut - Extract selected fields of each line of a file

diff - Compare two files

grep - Search text for a pattern

head - Display the first part of files

less - Display files on a page-by-page basis

sed - Stream editor (esp. search and replace)

sort - Sort text files

split - Split files

tail - Display the last part of a file

tr - Translate/delete characters

uniq - Filter out repeated lines in a file

wc - Line, word and character count

touch - Create a new empty file

Variable Herramienta que nos permite almacenar información de algún tipo, en particular puede ser: String, Integer, Array o Constantes.

Además, las variables ⌘ Variables de ambiente (globales y presentes en todos los shells) pueden ser... ⌘ Variables locales (definidas por el usuario)

para conocer las variables de ambiente se puede usar el comando env o printenv

⌘ PATH

⌘ USER

⌘ USERNAME

⌘ SHELL

Variables locales se definen como:

VARNAME="value"

Para usarlas en otros subshells se deberán exportar:

export VARNAME="value"

(se recomienda el uso de export)

Las variables son case sensitive.

Script

Un script es una secuencia de comandos y operaciones que el shell puede interpretar y ejecutar.

Esta secuencia de comandos se suele guardar en un archivo de texto con la extensión .sh. que luego se podrá ejecutar utilizando el comando bash, ., sh, etc. según el shell que estemos utilizando.

De esta forma podemos automatizar tareas.

Unscript debería:

- Ejecutarse sin errores
- Realizar la tarea para la que está destinado
- Tener la lógica del programa bien definida
- Evitar el código innecesario
- ser reusable

Crear con algún editor un archivo llamado **mi_script.sh**

Dentro del mismo, incluir el siguiente código:

```
#!/bin/bash

echo -n "Ingrese un nota: "
read x

if [ $x -ge 4 ]; then
echo "Ud. aprobó el examen"
else
echo "No aprobó el examen, su nota es menor a 4"
fi
```

Ejecutar: **bash mi_script.sh**

touch mi_primer_script.sh
nano mi_primer_script.sh

```
$ mi_primer_script.sh
1  #!/bin/bash
2
3  echo -n Ingrese un nota :
4  read x
5
6  if [ $x -ge 4 ]; then
7  echo Ud. aprobó el examen
8  else
9  echo No aprobó el examen, su nota es menor a 4
10 fi
11
```

Ejercicio 1

a. Crear la siguiente estructura de directorios:

Intro

Ejercicio1

Dentro de **Ejercicio1** crear el archivo un archivo llamado

datos_personales.txt

b. Listar el contenido para verificar que exista.

c. Editar el archivo creado y agregar:

```
28.764.999;Bruno;gutierrez;soltero;36
22.222.222;Alberto;Garcia;casado;55
33.333.333;Claudia;Riccio;soltera;40
55.555.555;Soledad;Villamil;soltera;24
```



```
mica@mica-VirtualBox:~/Escritorio$ mkdir Intro
mica@mica-VirtualBox:~/Escritorio$ ls
Intro
mica@mica-VirtualBox:~/Escritorio$ pwd
/home/mica/Escritorio
mica@mica-VirtualBox:~/Escritorio$ cd Intro
mica@mica-VirtualBox:~/Escritorio/Intro$ mkdir Ejercicio1
mica@mica-VirtualBox:~/Escritorio/Intro$ cd Ejercicio1
mica@mica-VirtualBox:~/Escritorio/Intro/Ejercicio1$ touch datos_personales.txt
mica@mica-VirtualBox:~/Escritorio/Intro/Ejercicio1$ nano datos_personales.txt
mica@mica-VirtualBox:~/Escritorio/Intro/Ejercicio1$ cat datos_personales.txt
```

Ejercicio 2

- Visualizar el contenido del archivo **datos_personales.txt**
- Realizar una copia del archivo **datos_personales.txt**, con el nombre **datos_personales_modif.txt**
- Modificar el archivo **datos_personales_modif.txt**, cambiando la palabra **soltero** por **casado**
- Contar la cantidad de letras del archivo **datos_personales.txt**

cat datos_personales.txt

cp datos_personales.txt datos_personales_modif.txt

sed -i 's/soltero/casado/g' datos_personales_modif.txt

wc -c datos_personales

Ejercicio 3

Crear un script que le pregunte al usuario por una **extensión**.

A continuación, el script debe evaluar la existencia de archivos con esa extensión y listarlos por pantalla.

```
20240308.zip
172823272.pdf
Apertura RI 2023.xlsx
Cool-Red-X-...sformed.png
```

xlsx

Apertura RI 2023.xlsx

```
#!/bin/bash
```

```
echo "Ingresa una extensión"
read extension
```

```
ls *.$extension
```

ls | grep '.\$extension'

find . -type -f -name '*.\$extension'

```
$ es_directorio_archivo.sh
1 #!/bin/bash
2 Parametro='$1'
3 ##### Revisar el parámetro #####
4 echo "Ingresa un archivo o directorio"
5 read parametro
```

```
6
7 if [ -d "$parametro" ]; then
8     for archive in "$parametro"/*; do
9         if [ -f "$archive" ]; then
10             echo "$archive"
11         fi
12     done
13 elif [ -f "$parametro" ]; then
14     cat "$parametro"
15 else
```

Ejercicio 4

Crear un script que reciba un parámetro y determine...

- Si es un directorio → que liste los archivos contenidos dentro del mismo
- Si es un archivo → que muestre por pantalla su contenido
- En otro caso → que muestre por pantalla el mensaje: **"No es archivo, ni directorio"**

Ejercicio 4

Crear un script que reciba un parámetro y determine...

- Si es un directorio → que liste los archivos contenidos dentro del mismo
- Si es un archivo → que muestre por pantalla su contenido
- En otro caso → que muestre por pantalla el mensaje: *"No es archivo, ni directorio"*

```
$ es_directorio_archivo.sh
1  #!/bin/bash
2  Parametro='$1'
3  ##### Recibir el parámetro #####
4  echo "Ingrese un archivo o directorio"
5  read parametro
6
7  if [ -d "$parametro" ]; then
8      for archive in "$parametro"/*; do
9          if [ -f "$archive" ]; then
10             echo "$archive"
11          fi
12      done
13  elif [ -f "$parametro" ]; then
14      cat "$parametro"
15  else
16      echo "No es archivo, ni directorio"
17  fi
```

Estructuras condicionales e iterativas: Shell como intérprete puede resolver estructuras condicionales e iterativas, facilitando y ampliando la forma de interactuar con el usuario y permitiendo un gran abanico de posibilidad a la hora de crear scripts.

```
if [[ CONDICIÓN ]]; then  
    echo "La condición se cumple, es True"  
  
else  
    echo "La condición no se cumple, es False"  
  
fi
```

Cómo Comparar.

Números:

-eq: Si son Iguales
-ne: Si no son iguales
-gt: Si el primero es mayor al segundo
-lt: Si el primero es menor al segundo
-ge: Si el primero es mayor o igual al segundo
-le: Si el primero es menor o igual al segundo

Cadenas:

string1 == string2: Si son Iguales
string1 != string2: Si no son iguales
string1 =~ regex: Devuelve True si la regex coincide
string1 > string2: Si la primer cadena es mayor que la segunda por orden léxico. Lo contrario para <
-z string: Devuelve True, si la longitud es cero
-n string: Devuelve True, si la longitud NO es cero

Case

```
case expression in  
    pattern1 )  
        statements ;;  
    pattern2 )  
        statements ;;  
    ...  
esac
```

Ejemplo

Crear un script que reciba como parámetro el nombre de un archivo y muestre un menú con las siguientes opciones.

- 1- Ver el contenido del archivo
- 2- Editar el archivo con el editor nano
- 3- Ver los permisos del archivo
- * - Salir

```
#!/bin/bash  
  
# Verifica si se proporcionó un argumento  
if [ $# -ne 1 ]; then  
    echo "Uso: $0 nombre_de_archivo"  
    exit 1  
fi  
  
archivo="$1"  
  
# Verifica si el archivo existe  
if [ ! -f "$archivo" ]; then  
    echo "El archivo '$archivo' no existe."  
    exit 1  
fi  
  
# Menú interactivo  
while true; do  
    echo ""  
    echo "Seleccione una opción:"  
    echo "1- Ver el contenido del archivo"  
    echo "2- Editar el archivo con el editor nano"  
    echo "3- Ver los permisos del archivo"  
    echo "*- Salir"  
    echo -n "Opción: "  
    read opcion  
  
    case "$opcion" in  
        1)  
            echo "Contenido de '$archivo':"  
            cat "$archivo"  
            ;;  
        2)  
            nano "$archivo"  
            ;;  
        3)  
            echo "Permisos de '$archivo':"  
            ls -l "$archivo"  
            ;;  
        *)  
            echo "Saliendo..."  
            break  
            ;;  
    esac  
done
```

Estructuras cíclicas

Nos permitirán repetir instrucciones dentro de un script

En Bash existen 3 estructuras: `for`, `while` y `until`. Cada una tiene un uso específico

Estructura For Nos permitirá iterar sobre una lista de valores donde

`i` es la variable que tomará valor

`[in LIST]` es la lista de valores.

Ejemplo:

```
in {1..5}
(( c=1; c<=5; c++ ))
in file1 file2 file3
in $(Linux-Or-Unix-Command-Here)
```

```
for i [in LIST];
```

```
do
```

```
COMMANDS;
```

```
done
```

Ejemplo For Crear un script que permita recorrer todos los archivos de un directorio y muestre el contenido de los que sean de extensión `.txt`

```
#!/bin/bash

# Verifica si se pasó un directorio como argumento
if [ $# -ne 1 ]; then
    echo "Uso: $0 directorio"
    exit 1
fi

directorio="$1"

# Verifica si el directorio existe
if [ ! -d "$directorio" ]; then
    echo "El directorio '$directorio' no existe."
    exit 1
fi

# Recorre todos los archivos .txt dentro del directorio
for archivo in "$directorio"/*.txt; do
    # Verifica que realmente exista al menos un archivo .txt
    if [ -f "$archivo" ]; then
        echo "Contenido de: $archivo"
        cat "$archivo"
        echo "-----"
    fi
done
```

Estructura While

```
while [ condition ]
```

```
do
```

```
comandos
```

```
done
```

```
until [condition]
```

```
do
```

```
comandos
```

```
done
```

Ejemplo de While y Until

crear un script utilizando `while` que permita imprimir el mensaje "Bienvenido por vez número" 5 veces

```
#!/bin/bash

contador=1

while [ $contador -le 5 ]; do
    echo "Bienvenido por vez número: $contador"
    contador=$((contador + 1))
done
```

Crear un script utilizando `until` que genere un número random entre 1 y 20, si los valores están entre 5 y 10 el bucle deberá terminar.

```
#!/bin/bash

number=0

until [[ $number -ge 5 && $number -le 10 ]]; do
    number=$((RANDOM % 20 + 1))
    echo "Número generado: $number"
done
```

Pipelines | Pipe o flujo de datos, nos permite actualizar la salida de un comando como entrada de otro

Un ejemplo sería `comando 1 | comando 2 | comando 3`

Lo que devuelva el comando 1 será utilizado por el comando 2 y la salida de este por el comando 3... así las veces que se desee.

Ejemplo Pipelines 1. contar la cantidad de líneas de un archivo de texto.

`cat ejemplo.txt | wc -l`

2 Localizar una palabra en un archivo

`cat ejemplo.txt | grep "Hola"`

3. Ordenar las líneas de un archivo

`cat ejemplo.txt | sort`

Búsqueda con grep con grep se puede buscar una palabra o patron y se imprimirá la línea o líneas que la contengan.

`grep opcion/es 'cadena_de_texto' fichero_donde_buscar`

Palabra o patrón a buscar

Archivo en el cual buscar

Ejemplos uso de grep

Buscar una palabra. `grep "Hola" ejemplo.txt`

Buscar una frase en un archivo: `grep "Hola mundo" ejemplo.txt`

Buscar múltiples palabras. `grep búsqueda1 archivo | grep búsqueda2 archivo`

Buscar una palabra en un conjunto de archivos: `grep -l palabra_a_buscar ./*`

Búsqueda y reemplazo Stream editor (sed)

`sed -i 's/old-text/new-text/g' input.txt`

- i indica hacer un update en el archivo

s. indica sustituir las palabras

g. indica hacerlo en forma global

Expresiones regulares

- ⊛ Una expresión regular es una cadena que sirve para expresar de forma compacta, un conjunto de cadenas
- ⊛ Son un potente mecanismo para resolver manipulación de textos y cadenas.
- ⊛ Uno de los comandos que aceptan RE es grep
- ⊛ Por otra parte hay múltiples lenguajes que las interpretan.

RE: Patrones básicos

BRE (Basic Regular Expression)

ERE (Extended Regular Expression) **La más usada**

PCRE (Perl Compatible Regular Expression)

Operación	Operador			Descripción	Ejemplo (ERE)
	BRE	ERE	PCRE		
Cuantificación	\?	?		Una vez o ninguna.	a?
	*			Las veces que sea (incluso ninguna)	a*
	\+	+		Al menos una vez	a+
	\{n,m\}	{n,m}		Entre n y m veces. Puede omitirse uno de los límites.	a{5,9}
Agrupación	Ver grupos.	(?regex)	(?:regex)	Para modificar el alcance de un operador	(?123)+
Alternativa	\			O lo uno o lo otro	(?B1as Luis)
Principio	^			El patrón comienza	^a
Fin	\$			El patrón acaba	a\$
Repr. universal	.			Cualquier carácter	.{2,3}
Escape	\			No interpretar un carácter especial	\.

RE: Patrones avanzados

Operación	Operador		Descripción	Ejemplo
	BRE	ERE/PCRE		
Alternativa	[...]		Uno de los caracteres incluidos. Puede indicarse un rango.	[A-Za-z]
	[^...]		Un carácter que no sea ninguno de los incluidos	[^A-Z]
Clases	Sin equivalencia	\w	Un carácter de palabra (letra, dígito o «_»).	\w+
	Sin equivalencia	\W	Un carácter que no sea de palabra.	^\W
	Sin equivalencia	\d	Un dígito, o sea, [0-9]	\d{4}
	Sin equivalencia	\D	Un carácter que no sea un dígito.	\D+
	Sin equivalencia	\s	Un carácter de espaciado, o sea, [\t\r\nf]	\w+\s\w+
	Sin equivalencia	\S	Un carácter que no sea de espaciado.	\S+\s
	Sin equivalencia	[:nombre:]	Un carácter de la clase nombre.	[[:alpha:];,;.]+
	Sin equivalencia	[=x=]	Cualquier variante del carácter «X»	[[: =a=]]
Límite de palabra	Sin equivalencia	\b[10]	Principio o fin de palabra.	\bdado\b
Grupos	\(regex\)	(regex)	Captura un grupo	(\w+)\s+\1
	\1, \2, ... \9		Refiere un grupo previamente capturado	

RE: Ejemplos

- ¿Contiene el texto una o ninguna letra 'a'? `grep -E 'a?' ejemplo.txt`
- ¿El texto contiene al menos dos letras 'z' seguidas? `grep -E 'z{2,}' ejemplo.txt`
- ¿Comienza el texto por una letra mayúscula? `grep -E '^[A-Z]' ejemplo.txt`
- La frase no contiene ningún dígito: `grep -E '^\D+$' ejemplo.txt`
- El uso de la opción **-E** nos permite colocar RE en grep.

Operadores de redirección

La salida estándar es la pantalla `stdout`, Pero se puede redirigir la misma a otro lugar, Por ejemplo, un archivo Para esto se utiliza el operador ">"

```
bruno@bruno:~$ date
```

```
bruno@bruno:~$ date > fecha.txt
```

```
bruno@bruno:~$ cat fecha.txt
```

→ Se redirige la fecha para ser guardada en el archivo fecha.txt.

El operador >, lo que hace es sobreescribir el archivo, mientras que el operador >> sirve como agregar al final del archivo, como si fuese un append.

Ejemplo: [damon@localhost ~]\$ date > ejemplo.txt

[damon@localhost ~]\$ hostname >> ejemplo.txt

Así como existen los operadores de salida, también existe el de entrada stdin,

Para esto se utiliza <

Ejemplo [damon@localhost ~]\$ sort < milista.txt Ordena el contenido de milista.txt,

[damon@localhost ~]\$ sort < mylist.txt > alphabetical-file.txt y lo muestra por pantalla

Por último existe un operador para stderr que es > 2 y >> 2

↓
mensajes de error (funciona como append)

Entonces se podría tener un archivo de log con errores como los ejemplos.

```
[damon@localhost ~]$ cat 2> error.log
```

```
[damon@localhost ~]$ cat 2>> error.log
```

Ejercicio 1: Solicitar al usuario el ingreso de los valores enteros y calcular el promedio de los mismos.

```
#!/bin/bash
contador=1
suma=0

while [ $contador -le 10 ]; do
    echo -n "Ingresá un número entero: "
    read valor

    suma=$((suma + valor))
    contador=$((contador + 1))
done

promedio=$((suma / 10))
echo "El promedio de los 10 valores ingresados es: $promedio"
```

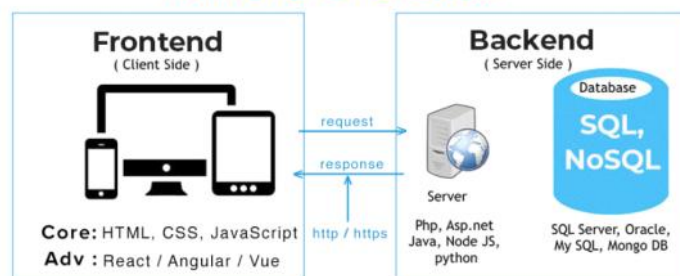

Introducción al desarrollo web

Desarrollo web

El desarrollo web es el proceso de crear y mantener sitios y aplicaciones web.

Abarca una amplia gama de tareas, desde el diseño y la planificación hasta la codificación y la implementación. El desarrollo web se suele separar entre Frontend y Backend

Frontend Vs Backend



Introducción al backend

El backend, también conocido como desarrollo del lado del servidor, se refiere a la parte de un sitio web o aplicación web que no es visible para los usuarios.

Se encarga de la lógica de la aplicación, el almacenamiento de datos y la comunicación con el servidor. Entre sus tareas se encuentran:

- ⊛ Implementar la lógica de la aplicación.
- ⊛ Almacenar y recuperar datos de una base de datos
- ⊛ Procesar solicitudes y enviar respuestas al usuario
- ⊛ Asegurar la seguridad del sitio web

Frameworks de Backend

Existen muchas herramientas para armar el backend de una aplicación o sitio web, dependiendo de la tecnología que se esté utilizando



Flask es un framework web para python. Es minimalista y se enfoca en proporcionar las herramientas básicas para construir aplicaciones web.

En lugar de incluir funcionalidades predefinidas como manejo de base de datos o validación de formularios, Flask permite a los desarrolladores elegir las librerías y herramientas adicionales que necesite para su proyecto.

Introducción al Frontend

El **frontend**, también conocido como desarrollo del lado del cliente, se refiere a la parte de un sitio web o aplicación web con la que los usuarios interactúan directamente.

Es la cara visible del sitio, y se encarga de la **interfaz de usuario (UI)** y la **experiencia de usuario (UX)**. Entre sus tareas se encuentran:

- ⊛ Diseñar la estructura y el layout de la página web
- ⊛ Implementar la lógica de la interfaz de usuario
- ⊛ Agregar elementos interactivos como botones, menús y formularios
- ⊛ Optimizar el rendimiento del sitio web para diferentes dispositivos

Lenguajes del Frontend



HTML (Hypertext Markup Language) es un **lenguaje de marcado** que define la estructura del contenido de una página web. Este lenguaje consiste en una serie de elementos que usarás para encerrar diferentes partes del contenido para que se vean o comporten de determinada manera. Importante: **HTML no es un lenguaje de programación**. Es un lenguaje de marcado.

Sirve para indicar una estructura de elementos, no para dar instrucciones a la máquina

CSS (Cascading Style Sheets) u Hojas de estilo en cascada es un **lenguaje de hojas de estilo**, es decir, te permite aplicar estilos de manera selectiva a elementos en documentos HTML.

Importante **CSS no es un lenguaje de programación**. Tampoco de marcado

Es un lenguaje de hojas de estilo. Sirve para dar estilo a elementos que ya fueron previamente indicados y ubicados en una estructura. Otros ejemplos de lenguajes de estilo son XSL y Sass.

JavaScript es un **lenguaje de programación** que se utiliza para añadir interactividad a las páginas web. Permite crear animaciones, juegos, formularios dinámicos y mucho más. Es uno de los lenguajes de programación más populares del mundo y se utilizan en millones de sitios web

```
const title = document.getElementById("titulo")

title.addEventListener("mouseover", () => {
  alert("Hola")
})

const changeColor = document.getElementById("change-color")

changeColor.onclick = function (event) {
  const R = randomNumber().toString()
  const G = randomNumber().toString()
  const B = randomNumber().toString()
  title.style.color = `rgb(${R},${G},${B})`
}

function randomNumber() {
  return (Math.random() * 256).toFixed(0)
}
```

Virtual environment

Para comenzar con este ejemplo utilizaremos un entorno virtual. Los entornos virtuales nos permiten instalar y utilizar librerías y la versión de Python necesaria para nuestro proyecto

Para esto utilizaremos PIPenv en nuestra carpeta del proyecto. Luego instalaremos la librería flask

Podemos verificar las dependencias instaladas. Entonces podemos comenzar nuestro proyecto.

Flask - Estructura básica

```
from flask import Flask

app = Flask(__name__)

if __name__ == "__main__":
    app.run(debug=True)
```

Este código nos permite levantar el servidor de nuestro sitio.

Por medio del parámetro debug podremos ver los cambios en tiempo real.

Por medio de funciones podremos renderizar contenido HTML tanto estática como dinámicamente.

```
@app.route('/')
def index():
    return "Hola mundo"
```

< > ↻ 127.0.0.1:5000

Hola mundo

```
<!DOCTYPE html>
<html>
  <head>
    <title>Mi página web</title>

    <!-- Inserto estilos de CSS -->
    <link rel="stylesheet" href="style.css">
  </head>
  <body>
    <h1 id="titulo">Titulo principal</h1>

    <!-- Cuerpo de nuestra página Web -->

    <!-- Inserto funcionalidad de Javascript -->
    <script src="script.js"></script>
  </body>
</html>
```

HTML - Ejemplo de una página

Flask - Templates y Jinja

Flask nos permite utilizar y renderizar templates (básicamente archivos HTML que sirven de base a lo que se quiera mostrar)

Para esto es necesario cargar los archivos HTML necesarios a la carpeta templates dentro del proyecto, e importar la función render_template. A partir de esta función también es posible pasar datos para que se rendericen.

Jinja nos permite agregar funcionalidad a estos templates, haciendo que puedan integrar data desde el template dinámicamente.

Flask-routing con Flask es sencillo definir el enrutamiento de distintas páginas por medio de decoradores de Python que definan la ruta.

```
@app.route('/')
def index():
    return "Hola mundo"
```

```
@app.route('/about')
def about():
    return render_template('about.html')
```

Flask

Es un framework minimalista web muy popular que deja casi todas las decisiones de diseño y arquitectura en manos del desarrollador.

Para trabajar con distintos proyectos es necesario crear un ambiente virtual (virtual environment), esto permitirá que no haya incompatibilidades entre diferentes versiones de dependencias instaladas entre un proyecto y otro `pipenv` <https://docs.python-guide.org/dev/virtualenvs/>

Creando un Proyecto Flask

1. crear un directorio donde usaremos el proyecto `mkdir mi_proyecto`
2. Acceder al directorio `cd -mi.proyecto`
3. Ingresar a Visual Studio Code `code .` (crear el archivo `app.py` y un directorio `.venv`)
4. crear el ambiente virtual `pipenv install flask`
- Se instalarán las dependencias en los archivos `pipfile` y `pipfile.lock`
5. Activamos el proyecto `pipenv shell`
6. Continuamos configurando el proyecto dentro del archivo `app.py` `from Flask import Flask`
7. Una vez dentro del entorno, podemos utilizar comandos de `pip` y `flask` normalmente,

❁ `pip list` ❁ `pip install flask` ❁ `pip --version` ❁ `flask run`

Arrancando el proyecto

Ejemplo de nuestro archivo `app.py`

```
from flask import Flask

app = Flask(__name__)

@app.route('/')
def hello_world():
    return 'Hello, World!'

if __name__ == '__main__':
    # esto solo funciona usando python3 app.py
    app.run("localhost", port=8081, debug=True)
```

Para ejecutar la app de Flask hay 3 formas:

```
flask run

flask run --host=127.0.0.1 --port=8081 --debug

python3 app.py
```

```
app.py > ...
1  from flask import Flask, render_template, request, redirect, url_for
2
3  app = Flask(__name__)
4
5  @app.route("/")
6  def hello_msg():
7      return render_template('home.html', nombre="Bruno")
8
9
10 if __name__ == "__main__":
11     app.run("127.0.0.1", port="8080", debug=True)
```

} Inicialización de Flask en nuestro Proyecto

Decorador `@`, indicando la ruta de acceso a la función

Setup del servidor

Render de Jinja para abrir la vista del template

Estructura básica de un proyecto (sin Blueprints)

/mi-proyecto

app.py

/venv

/static

style.css

/templates

layout.html

index.html

login.html

...

Jinja nos permite agregar funcionalidad a estos templates,

haciendo que puedan integrar data desde el template

dinámicamente.

Templates :: Variables y estructuras Dentro de un template, además de tener código HTML, también podremos

incorporar en forma dinámica variables y estructuras tipo if o for, entre muchas cosas más

• Variables

Se incorporarán en forma dinámica su valor cuando las llamemos.

`{{ nombre_variable }}`

Variables dentro de los Templates Los templates pueden soportar muchos tipos de variables,

Por ejemplo: Diccionarios, listas, cadenas, objetos, enteros

`<p>A value from a dictionary: {{ mydict['key'] }}.</p>`

`<p>A value from a list: {{ mylist[3] }}.</p>`

`<p>A value from a list, with a variable index: {{ mylist[myintvar] }}.</p>`

`<p>A value from an object's method: {{ myobj.somemethod() }}.</p>`

Estructuras IF dentro de los templates

como dijimos también pueden existir estructuras de tipo condicionales if

Las mismas deberán estar entre `{% ... %}`

```
{% if user %}
    Hello, {{ user }}!
{% else %}
    Hello, Stranger!
{% endif %}
```

Estructuras for dentro de los templates

También se podrían utilizar for para hacer iterar una lista o diccionario de datos

Las mismas deberán estar entre `{% ... %}`

```
<ul>
    {% for comment in comments %}
        <li>{{ comment }}</li>
    {% endfor %}
</ul>
```

Links dentro de los templates

❁ Los links dentro de los templates se recomienda que se use el método `url_for()` de esta forma evitaremos posibles problemas o roturas de nuestro código.

❁ Los links pueden ser estáticos o dinámicos

Ejercicio

Continuar con este mismo proyecto, y en la vista donde escribieron hola mundo, incorporar debajo un link que lleve a una nueva vista llamada howiam.html que tenga un texto con una breve reseña del alumno.

```
from flask import Flask, render_template

app = Flask(__name__)

@app.route('/')
def hola_mundo():
    return render_template('index.html')

@app.route('/howiam')
def howiam():
    return render_template('howiam.html')
```

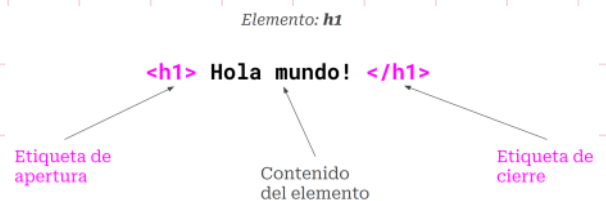
```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
</head>
<body>
  <h1>Hola mundo</h1>
  <a href="/howiam">Ver información</a>
</body>
</html>
```

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
</head>
<body>
  <p>What is Lorem Ipsum?
  Lorem Ipsum is simply dummy text of the printing and typesetting industry. Lorem Ips
</p>
</body>
</html>
```

Estructura básica de HTML

```
<!DOCTYPE html>  Declaración de tipo de documento HTML
<html>           Inicio documento HTML
  <head>
    <title>Mi página web</title> Cabecera de la página
  </head>
  <body>
    <h1>Hola mundo!</h1>  Cuerpo o contenido de la página
  </body>
</html>           Fin documento HTML
```

Etiquetas



Etiquetas básicas

- `<html>`, `<head>`, `<body>` para la estructura básica de la página Web
- `<h1>`, `<h2>`, ..., `<h6>` para títulos dentro del contenido
- `<div>` para dividir el contenido en partes con significado distinto
- `<p>` para párrafos
- `<img>` para añadir imágenes al documento
- `<a>` para enlaces
- `<br>` para realizar saltos de línea
- `<span>` para estilizar una parte del texto
- `<form>`, `<input>`, `<button>`, `<label>` para formularios

Atributos

Los atributos permiten modificar propiedades de un elemento HTML

Siempre se escriben dentro de la etiqueta de apertura `<a href="http://google.com/"> Ir a Google </a>`

Atributo o propiedad

Valor

Ejemplo de una página web básica

Estructura básica de CSS



```
<!DOCTYPE html>
<html lang="es">
<head>
  <title>Pala Store</title>
</head>
<body>
  <h1 class="titulo">Pala Store: PCs a tu medida</h1>
  <div id="productos">
    <h2 class="titulo">Nuestros productos</h2>
    <ul id="lista-componentes">
      <li>Procesador Ryzen 9 7950x3d</li>
      <li>Placa de video Nvidia RTX 4090 24gb</li>
      <li>Motherboard Gigabyte X670E Aorus Master</li>
    </ul>
    <button id="boton-ver-mas">Ver más</button>
  </div>
  <footer>
    <p>Mail de contacto: pala_store_clientes@example.com</p>
    <p>Pala Store 2024</p>
  </footer>
</body>
</html>
```

Selectores

Los selectores nos indican con base en qué criterio CSS selecciona qué elementos de nuestra página va a tomar para aplicarle un conjunto de atributos.

tipos de selectores

- Selector por **ID (#)**
- Selector por **clase (.)**
- Selector por **etiqueta o tipo** (ej. body)
- Selector por **atributo** ([attr=value])
- Selector por **pseudo-clase (:)**
- Selector **universal (*)**

Atributos

Los atributos CSS se refieren a la característica visible o estructural que queremos modificar de un elemento por ejemplo: el color del texto o la forma de mostrar un botón con respecto al resto de la página

Algunos atributos básicos son:

Textos: font-family, font-size, font-style, color

Elementos: display, padding, margin, border-radius, width, height, opacity, flex

```
body, button {
  font-family: Cambria;
  background-color: #323232;
}

.titulo {
  margin-bottom: 1.5em;
  color: violet;
}

#productos, footer {
  text-align: center;
}

li {
  list-style-type: none;
  margin-bottom: 2vh;
}

#boton-ver-mas {
  font-size: 16px;
  border-radius: 10px;
  width: 75px;
  height: 30px;
  margin-top: 4vh;
}

footer {
  margin-top: 30px;
  border-top: gray 2px solid;
  color: gray;
}

button:hover {
  cursor: pointer;
}

#productos, button {
  color: rgb(245, 64, 121);
}
```

Insertando CSS a una página web

Existen 3 formas de insertar código CSS a una página web.

- Inline: inserto el estilo directamente como atributo del elemento HTML.

```
<h1 style="color: red;">Hola mundo!</h1>
```

- Dentro del HTML: etiqueta <style> dentro de la cabecera del documento

```
<!DOCTYPE html>
<html>
<head>
  <title>Mi página Web</title>
  <style>
    h1 {
      color: red;
    }
  </style>
</head>
<body>
  <h1>Hola mundo!</h1>
</body>
</html>
```

- Desde un archivo CSS: lo enlazamos dentro de la cabecera del documento HTML.

```
<!DOCTYPE html>
<html>
<head>
  <title>Mi página Web</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <h1>Hola mundo!</h1>
</body>
</html>
```

Ejercitación

Ejercicio: Pala Store: Tomar el ejemplo de esta PPT de "Pala store" y agregarle imágenes a los productos. Las imágenes tienen que tener un tamaño de 150 px x 150 px
Mostrar la página usando Flask

pueden usar tanto el HTML como el CSS de ejemplo

```
<!DOCTYPE html>
<html lang="es">
<head>
  <title>Pala Store</title>
</head>
<body>
  <h1 class="titulo">Pala Store: PCs a tu medida</h1>
  <div id="productos">
    <h2 class="titulo">Nuestros productos</h2>
    <ul id="lista-componentes">
      <li>Procesador Ryzen 9 7950x3d</li>
      
      <li>Placa de video Nvidia RTX 4090 24gb</li>
      
      <li>Motherboard Gigabyte X670E Aorus Master</li>
      
    </ul>
    <button id="boton-ver-mas">Ver más</button>
  </div>
  <footer>
    <p>Mail de contacto: pala_store_clientes@example.com</p>
    <p>Pala Store 2024</p>
  </footer>
</body>
</html>
```

Javascript

Javascript es un lenguaje de programación que se utiliza para añadir interactividad a las páginas web.

Permite crear animaciones, juegos, formularios dinámicos y mucho más. Es uno de los lenguajes de programación más populares del mundo y se utilizan en millones de sitios web.

A su vez, también es utilizado como lenguaje para el backend, a través de aplicaciones de servidor como por ejemplo Node.js. Nos basaremos en el estándar EcmaScript 2015 (ES6).

El punto y coma (;) al final de cada instrucción es optativo.

Variables y constantes

`const` PI = 3.14159265359 ← **constante**

`let` alumnosCursando = 237 ← **variable**

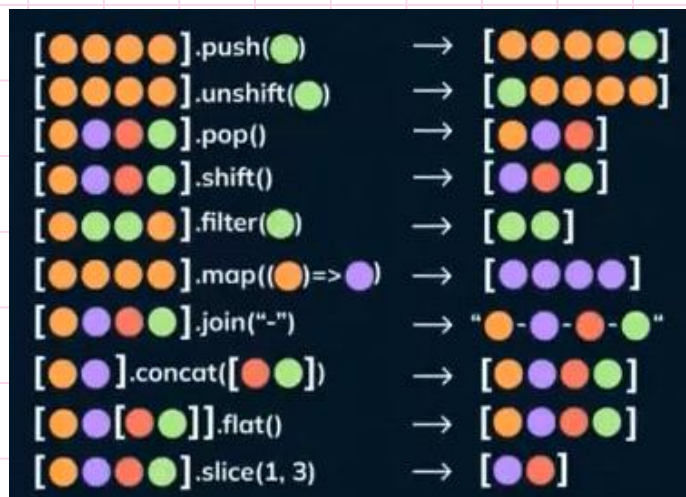
PI = 3

TypeError: Assignment to constant variable.

Tipos de datos

```
const alumnos = 237 // number
const precio = 1392.50 // number
const nombre = "Bruno" // string
const esDomingo = false // boolean
const persona = {nombre: "pepe", edad: 26} // object
const numeros = [1, 2, 3] // object
let valorInicial = null // object
const ultimoDigitoDePI = undefined // undefined
```

Podemos preguntar el tipo de dato usando el operador `typeof`



Estructuras de control

```
let numero = 8

if (numero < 20) {
  console.log("El número es menor a 20")
} else if (numero == 20) {
  console.log("El número es igual a 20")
} else {
  console.log("El número es mayor a 20")
}

while (numero < 20) {
  numero++ // numero = numero + 1
}

for (let i=1; i<5; i++) {
  const nuevoValor = numero + i
  console.log("Ahora el número vale: " + nuevoValor)
}
```

```
const color = "red"

switch(color) {
  case "red": {
    console.log("El color es rojo")
    break
  }
  case "blue": {
    console.log("El color es azul")
    break
  }
  case "yellow": {
    console.log("El color es amarillo")
    break
  }
  default: {
    console.log("El color no existe")
  }
}
```


Concatenación de Strings

Hay dos formas principales de concatenar cadenas:

```
const nombre = "Pepito"
const apellido = "Perez"

// Usando el operador +
const fullName1 = nombre + " " + apellido

// Usando template literals
const fullName2 = `${nombre} ${apellido}`

console.log(fullName1) // Pepito Perez
console.log(fullName2) // Pepito Perez
```

Se usan **backticks** (') en vez de comillas normales!

Listas / Arrays

Ejemplo de cómo recorrer listas usando JavaScript.

```
const nombres = ["Juan", "Pedro", "María", "Ana", "Luis"]

// Usando un bucle for
for (let i = 0; i < nombres.length; i++) {
  console.log(nombres[i])
}

// Usando un bucle forEach
nombres.forEach(function(nombre) {
  console.log(nombre)
})

// Usando un bucle for...of
for (const nombre of nombres) {
  console.log(nombre)
}
```

Nota: el operador **in** se usa para objetos y el **of** para listas!

Objetos

Los objetos en JavaScript son "similares" a los

diccionarios de Python. Sin embargo, su entendimiento al

completo depende de conocimientos de Paradigmas de Programación

uso en esta materia se limitará al manejo de JSON

Keywords: (compilador)

Ejemplos: const, let, var, number, string, object, null, undefined, function, if, else, true, false, return, for, break, continue, while, switch, this, in, length, document

Funciones

Función clásica

palabra reservada function nombre de la función argumentos

```
function modulo(x, y) {
  const sumaCuadrados = x*x + y*y
  const resultado = Math.sqrt(sumaCuadrados)
  return resultado
}
```

Cuerpo de la función

retorno de la función

Funciones flecha

```
// (args) => { functionBody }
const modulo = (x, y) => {
  return Math.sqrt(x*x + y*y)
}

// (args) => returnValue
const suma = (x, y) => x + y

console.log(typeof suma) // function
console.log(typeof modulo) // function
console.log(suma(2, 3)) // 5
console.log(modulo(4, 6).toFixed(2)) // 7.21
```

Funciones como tipo de dato

En JavaScript, las funciones son **first-class citizens**, es decir, se pueden manipular como valores (como si fuera un string, por ejemplo)

```
const numeros = [1, 2, 3, 4, 5, 6] // función anónima
const filtrados = numeros.filter(x => x % 2 == 0)

console.log(filtrados) // resultado: [2, 4, 6]
```

En este ejemplo, estamos pasando una función para saber si un número es par como valor (por parámetro) al método filter, quien internamente la va a llamar para filtrar los números de nuestra lista

Funciones como argumento También pueden formar parte del argumento de otra función

```
function student(){
  return "Alumno";
}

function greet(user){
  console.log("Bienvenido", user());
}

// Prints "Bienvenido Alumno"
let message = greet(student);
```

Funciones como resultado de otra función

Las funciones también pueden ser devueltas en el "return" por otra función.

```
const greet = function (valor) {
  return function () {
    console.log("Bienvenido a nuestra materia,", valor);
  }
}

greet("Juan")();
```

Bonus track : fetch

```
const pokemonURL = "https://pokeapi.co/api/v2/pokemon?limit=3&offset=0"

fetch(pokemonURL)
  .then(response => response.json())
  .then(json => console.log(json))
```

Bonus del bonus:
tipo de dato Promise

```
{
  count: 1302,
  next: 'https://pokeapi.co/api/v2/pokemon?offset=3&limit=3',
  previous: null,
  results: [
    { name: 'bulbasaur', url: 'https://pokeapi.co/api/v2/pokemon/1/' },
    { name: 'ivysaur', url: 'https://pokeapi.co/api/v2/pokemon/2/' },
    { name: 'venusaur', url: 'https://pokeapi.co/api/v2/pokemon/3/' }
  ]
}
```

Integración con HTML y CSS

DOM: modificando nuestra página web desde JavaScript

El **DOM (Document Object Model)** es una interfaz de programación para representar elementos HTML como objetos en JavaScript. Permite la manipulación de nuestra página web a través del código.

- Cada elemento HTML es un objeto en JavaScript
- Cada atributo de ese elemento HTML es una propiedad de ese objeto en JavaScript.
- Los elementos HTML responden a eventos. Esos eventos en JavaScript se representan como funciones asignadas al objeto

¿Cómo se usa el DOM?

Ejemplo: obtener el contenido del segundo párrafo

HTML:

```
<p>Hola mundo!</p>
<p>¡Aguante FIUBA!</p>
```

En JavaScript:

```
const parrafos = document.getElementsByTagName("p")
const textoParrafo2 = parrafos[1].innerText
console.log(textoParrafo2)
```

Palabra reservada: el objeto que
representa nuestro documento HTML

¿qué puede hacer el DOM con JavaScript?

- puede cambiar todos los elementos HTML en la página
- " atributos "
- " estilos CSS "
- puede eliminar elementos y atributos HTML existentes
- puede agregar nuevos elementos y atributos HTML
- puede reaccionar a todos los eventos HTML existentes en la página
- puede crear nuevos eventos HTML en la página.

Algunas funciones y props. básicas del DOM

- `document`. `getElementsByTagName(...)`
- `document`. `getElementById(...)`
- `document`. `getElementsByClassName(...)`
- `document`. `createElement(...)`
- `document`. `body`

- elemento. `append(...)`
- elemento. `appendChild(...)`

- elemento. `innerText`
- elemento. `innerHTML`
- elemento. `style`
- elemento. `parentElement`

Eventos: nuestra página responde a acciones del usuario

¿qué pasa si hago click en un botón? Como responde nuestra página ante este suceso es lo que se conoce como un evento.

Para definir la respuesta a un determinado evento de nuestra página web necesitamos:

- Un objeto del DOM que "escuche" el evento (por ejemplo, un button)
- Una función asignada a ese objeto que realice una acción cada vez que el objeto reciba una señal de ese evento.

Escuchando un evento

Ejemplo: mostrar una alerta cada vez que se haga click en un botón

HTML: `<button id="alerta">Pulse aquí!</button>`

```
JS:
const botonAlerta = document.getElementById("alerta")
botonAlerta.addEventListener("click", function(event) {
  alert("Hola mundo!")
})

const botonAlerta = document.getElementById("alerta")
botonAlerta.onclick = function(event) {
  alert("Hola mundo!")
}
```


Importando Scripts en nuestra página web

Existen dos formas de importar código JavaScript a nuestro documento HTML :

⊗ Insertando el código dentro del elemento `<script>`

⊗ Enlazando un archivo .js en el atributo src de la etiqueta `<script>`

la etiqueta `<script>` se coloca al final del body.

```
<!DOCTYPE html>
<html lang="es">
<head>
  <title>Mi página Web</title>
</head>
<body>
  <button id="alerta">Pulse aquí!</button>
  <script>
    const botonAlerta = document.getElementById("alerta")
    botonAlerta.addEventListener("click", function(event) {
      alert("Hola mundo!")
    })
  </script>
</body>
</html>
```

```
<!DOCTYPE html>
<html lang="es">
<head>
  <title>Mi página Web</title>
</head>
<body>
  <button id="alerta">Pulse aquí!</button>
  <script src="script.js"></script>
</body>
</html>
```

Ejemplo con Flask

Creamos nuestra app

```
from flask import Flask, render_template

app = Flask(__name__)

@app.route('/')
def index():
    return render_template("index.html")

if __name__ == '__main__':
    app.run(host="localhost", port=8080, debug=True)
```

¿Dónde va el archivo
index.html?

Tenemos el siguiente documento HTML...

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width,
initial-scale=1.0">
  <title>Document</title>
</head>
<body>
  <h1 id="titulo">Hola mundo!</h1>
  <button id="change-color">Cambiar color</button>
  <script src="script.js"></script>
</body>
</html>
```

Vamos a incorporar un script de JavaScript
con la etiqueta `<script>`

en Flask se utiliza url_for

... y un script de JS vinculado al HTML

```
const title = document.getElementById("titulo")
title.addEventListener("mouseover", () => {
  alert("Hola")
})

const changeColor = document.getElementById("change-color")
changeColor.onclick = function(event) {
  const R = randomNumber().toString()
  const G = randomNumber().toString()
  const B = randomNumber().toString()
  title.style.color = `rgb(${R},${G},${B})`
}

function randomNumber() {
  return (Math.random() * 256).toFixed(0)
}
```

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
</head>
<body>
  <h1 id="titulo">Hola mundo!</h1>
  <button id="change-color">Cambiar color</button>
  <script src="{{ url_for('static', filename='js/script.js') }}"></script>
</body>
</html>
```

Los archivos js, css, imágenes, fuentes
van en la carpeta static

Ejercicio Crear una página web usando Flask que contenga 2 campos de texto y un botón.

El botón debe ser rojo con letra blanca. Al pulsar dicho botón se deberá mostrar una alerta por pantalla que muestre la concatenación de las cadenas que se ingresen

en los campos de texto. 🌟 Pista 1 : existe un elemento HTML del tipo "campo de texto" .

🌟 Pista 2 : existe un atributo del DOM que me devuelve el contenido ingresado en un campo de texto.

```
body {
  font-family: sans-serif;
  padding: 20px;
}

input {
  padding: 10px;
  margin: 5px;
  border-radius: 5px;
  border: 1px solid #ccc;
}

button {
  padding: 10px;
  margin: 5px;
  border-radius: 5px;
  border: 1px solid #ccc;
  background-color: red;
  color: white;
  cursor: pointer;
}
```

```
function concatenar() {
  const texto1 = document.getElementById("campo1").value;
  const texto2 = document.getElementById("campo2").value;
  const resultado = texto1 + texto2;
  alert("El resultado es: " + resultado);
}
```

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
  <link rel="stylesheet" href="{{ url_for('static', filename='style.css') }}">
</head>
<body>
  <h2>Concatenar textos</h2>
  <input type="text" id="campo1" placeholder="Texto 1">
  <input type="text" id="campo2" placeholder="Texto 2">
  <button onclick="concatenar()">Concatenar</button>

  <script src="{{ url_for('static', filename='script.js') }}"></script>
</body>
</html>
```

Backend

El back end (o backend), también conocido como desarrollo del lado del servidor, se refiere a la parte de un sitio web o aplicación web que no es visible para los usuarios. Se encarga de la lógica de la aplicación, el almacenamiento de datos y la comunicación con el servidor.

Entre sus tareas se encuentran:

- ⊛ Implementar la lógica de la aplicación
- ⊛ Almacenar y recuperar datos de una base de datos
- ⊛ Procesar solicitudes y enviar respuestas al usuario
- ⊛ Asegurar la seguridad del sitio web

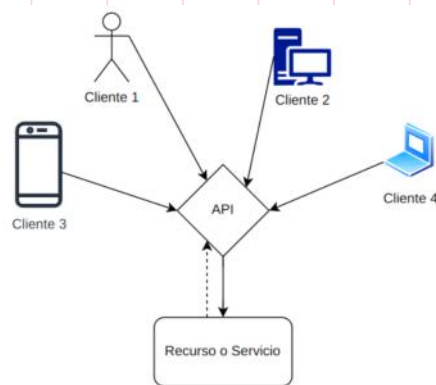
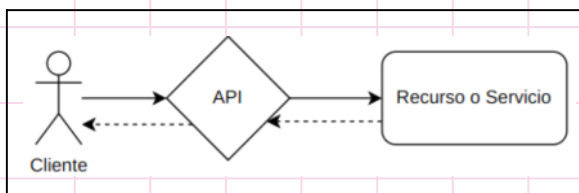
APIs

Una API o una interfaz de programación de aplicaciones es un conjunto de definiciones y protocolos que se usa para diseñar e integrar el software de las aplicaciones.

Las APIs permiten que sus productos y servicios se comuniquen con otros, sin necesidad de saber cómo están implementados.

En nuestro caso utilizaremos, una API para comunicar el frontend de la aplicación con los datos almacenados en el back end.

Función de una API



Tipos de APIs - Según su política de uso

- ⊛ APIs públicas o abiertas están disponibles para que otros usuarios o desarrolladores las empleen con mínimas restricciones o, en algunos casos incluso, están totalmente accesibles.
- ⊛ APIs privadas o internas están ocultas de los usuarios externos y se exponen únicamente para los sistemas internos de una organización. Se emplean para el desarrollo interno de la empresa, optimizando la productividad y la reutilización de servicios.
- ⊛ APIs de aliados comerciales o partners son aquellas que se exponen entre los membros de una alianza comercial. Como no están disponibles para todos, se necesita una autorización especial para usarlas.
- ⊛ APIs compuestas utilizan distintos datos o diversas APIs de servicio y permiten que los desarrolladores puedan acceder a varios terminales.
Según su uso:
 - ⊛ API de datos proporcionan a varios bancos de datos o proveedores SaaS (Software as Service) acceso CRUD (Create, Read, Update, Delete) a conjuntos de datos, permitiendo la comunicación entre una aplicación y un sistema de gestión de bases de datos.
 - ⊛ API de sistemas operativos definen cómo las aplicaciones usan los recursos disponibles y servicios del sistema operativo. Por lo que cada OS posee un conjunto de APIs.
 - ⊛ APIs remotas define los estándares de interacción que las aplicaciones tienen en diferentes dispositivos, es decir, un software accede a ciertos recursos ubicados fuera del dispositivo que los solicita, como dice su nombre. Como dos aplicaciones se conectan de forma remota a través de una red, las APIs remotas usan protocolos para lograr la conexión.
 - ⊛ APIs web esta clase de APIs es la más común, dado que las APIs web proporcionan datos que los dispositivos pueden leer y transferirlos entre sistemas basados en la web o arquitectura cliente - servidor.

o arquitectura cliente - servidor.

Protocolos en APIs web

Los protocolos de la API permiten estandarizar el intercambio de datos entre los diferentes servicios web. Esto brinda la oportunidad de acceder a capacidades en diversos sistemas, a través de diferentes lenguajes de programación y en distintos sistemas operativos.

⊗ RCP - Remote Procedure Call

⊗ SOAP - Service Object Protocol

⊗ **REST - Representational State Transfer**

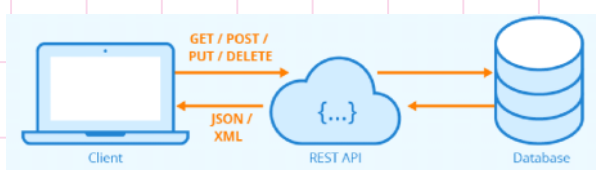
⊗ GraphQL

REST - Representational State Transfer

REST es una interfaz para conectar varios sistemas basados en el protocolo HTTP (uno de los protocolos más antiguos) y nos sirve para obtener y generar datos y operaciones, devolviendo esos datos en formatos muy específicos, como XML y Json. Sus principios más importantes son:

- ⊗ Un protocolo cliente / servidor sin estado: cada mensaje HTTP contiene toda la información necesaria para comprender la petición. Como resultado, ni el cliente ni el servidor necesitan recordar ningún estado de las comunicaciones entre mensajes.
- ⊗ Un conjunto de operaciones bien definidas que se aplican a todos los recursos de información. Ejemplos. POST, GET, PUT y DELETE
- ⊗ Una sintaxis universal para identificar los recursos. En un sistema REST, cada recurso es direccionable únicamente a través de su URL.
- ⊗ El uso de hipertextos: es posible navegar de un recurso REST a muchos otros simplemente siguiendo enlaces.

Restful API o API REST



Entonces una **RESTful API** es una API que sigue los **principios de REST**

Los principios RESTful se basan en la idea de que los recursos se representan como objetos y que las operaciones sobre esos recursos se realizan utilizando métodos HTTP estándar. Esto hace que las APIs RESTful sean fáciles de consumir y desarrollar.

Partes principales de una REST API

Request

- ⊛ URI
- ⊛ Verbos HTTP
- ⊛ Headers
- ⊛ Request Body

Response

- ⊛ Código de estado
- ⊛ Response Body
- ⊛ Response time

Request - Petición al servidor

Para realizar una petición necesitamos una serie de datos.

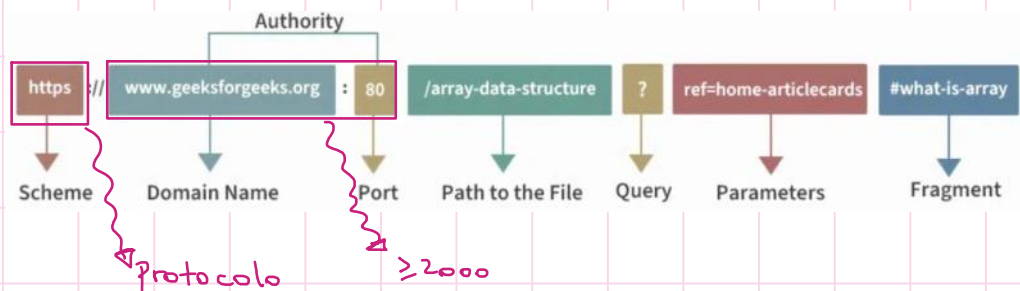
El objeto request debe estar formado con una serie de elementos:

- ⊛ Verbo o Método HTTP
- ⊛ URI (Uniform Resource Identifier)
- ⊛ Headers
- ⊛ Objeto body (en caso de que corresponda)

URI/URL El URI (Uniform Resource Identifier) sirve para acceder a un recurso

físico o abstracto por Internet. Dependiendo de la situación, el recurso puede ser de muchos tipos: por ejemplo, una página web o el remitente o al destinatario de un correo electrónico. También se lo conoce como URL (uniform resource locator)

Partes que
componen al
URI:



Verbos / métodos HTTP

El protocolo HTTP define un conjunto de métodos de petición para indicar la acción que se desea realizar para un recurso determinado.

Los verbos HTTP más comunes para realizar peticiones son

<u>GET</u>	<u>Obtiene un recurso</u>
<u>POST</u>	<u>Crear un nuevo recurso</u>
<u>PUT</u>	<u>Modifica los datos</u>
<u>DELETE</u>	<u>Elimina un recurso</u>
<u>PATCH</u>	<u>Modifica un recurso en particular</u>

Response - Respuesta del servidor

El objeto response se utiliza para especificar el código de estado de la respuesta, los encabezados y el cuerpo de la respuesta.

El response time es el tiempo en que el servidor tarda en responder a un request.

Este tiempo en ocasiones es vital y por lo tanto debe optimizarse el código (brindar un SLA). En nuestro caso esto no es necesario.

Códigos de estado HTTP El código de estado (status code) de la respuesta es un número que indica el resultado de la solicitud.

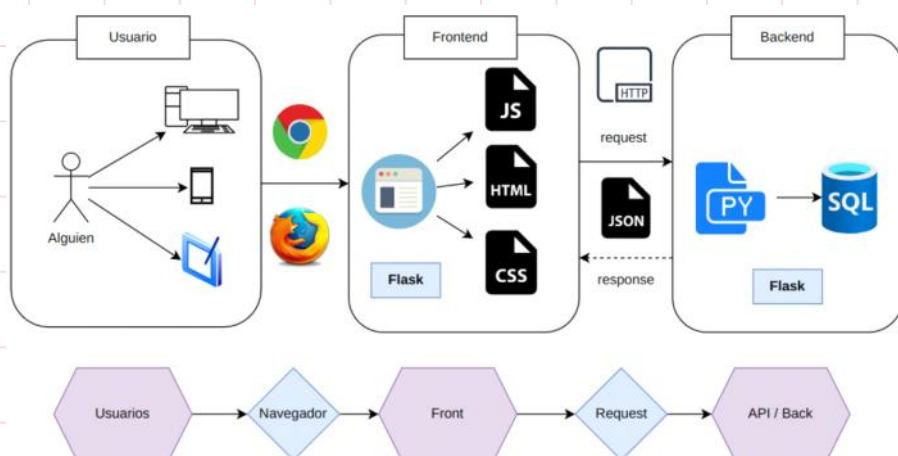
Code	Significado
1xx	Informativas
2xx	Correctas
3xx	Redirecciones
4xx	Errores del cliente
5xx	Errores del servidor

Ejemplos:

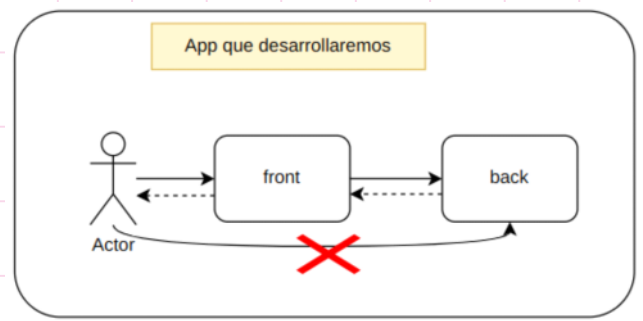
Code	Significado
200	OK
201	Created
400	Bad Request
401	Authorization Required
403	Forbidden
404	Not Found
500	Internal Server Error
504	Service Temporarily Unavailable

Integración entre "front" y "back"

Resumen niveles de Desarrollo web



¿Cómo vamos a trabajar nosotros con las APIs?



Veamos cómo se ve en Flask

Herramientas

Postman - Link Postman es una herramienta de colaboración y desarrollo que permite a los desarrolladores interactuar y probar el funcionamiento de servicios web y aplicaciones, proporcionando una interfaz gráfica intuitiva y fácil de usar para enviar solicitudes a servidores web y recibir las respuestas correspondientes.

Con esta plataforma se puede gestionar diferentes entornos de desarrollo, organizar las solicitudes en colecciones y realizar pruebas automatizadas para verificar el comportamiento de los sistemas



Testing

En desarrollo de software, el testing (o pruebas de software) es el proceso de evaluar y verificar que un producto o aplicación de software cumple con su función. El testing sirve entre otras cosas para:

- ⊛ Validar que la funcionalidad de un programa sea la esperada
- ⊛ Optimizar el rendimiento de la aplicación
- ⊛ Encontrar y corregir fallas de seguridad (vulnerabilidades)

Existen distintos tipos de pruebas de software. Las más comunes son:

⊛ Pruebas unitarias: se verifica que una sola funcionalidad (feature) de todas las que existen en nuestro programa funcione correctamente para todos los casos de uso posibles.

⊛ Pruebas de integración: se prueban varias features juntas comunicándose entre sí. Se verifica que el resultado final de todo ese proceso sea el esperado, idealmente para todos los casos de uso posibles.

⊛ Pruebas de estrés: se expone al programa a un conjunto de escenarios donde se ponen al límite las capacidades técnicas del mismo (tiempos de respuesta, velocidad de transferencia de archivos, capacidad de procesamiento de peticiones HTTP, etc)

Pruebas unitarias (Unit tests) Son una técnica de testeo que consiste en probar unidades o componentes individuales de un software de forma aislada.

Por lo general se prueban funciones o métodos

Para realizar una prueba unitaria, dado un conjunto de datos de entrada, se verifica que los datos de salida correspondan a la lógica prevista por el desarrollador.

Ejemplo de Pruebas unitarias

```
function sumar(a, b) { return a + b }  
const suma = sumar(num1, num2)
```

- Unit test 1: si $num1 = -num2$ entonces $suma = 0$
- Unit test 2: si $num1 = 0$ y $num2 = x$ entonces $suma = x$
- Unit test 3: si $num1 \neq 0$ y $num2 = x$ entonces $suma = num1 + x$
- Unit test 4: si $num1 = 0$ y $num2 = -x$ entonces $suma = -x$

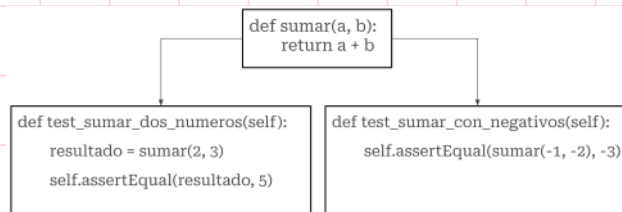
Pruebas de Integración (Integration tests) Son pruebas que verifican que varios componentes del sistema funcionen correctamente cuando se combinan.

Los tests de integración se centran en cómo interactúan las distintas partes del sistema entre sí

- Esto puede incluir:
- ⊛ Rutas (endpoints) que llaman a funciones internas.
 - ⊛ Procesamiento de datos y respuestas HTTP
 - ⊛ Interacción con bases de datos, APIs externas, etc

Ejemplo con Flask

Ejemplo de test unitario



Ejemplo test de integración

```
app = Flask(__name__)  
@app.route('/ping')  
def ping():  
    return jsonify("pong"), 200
```

```
class TestRoutes(unittest.TestCase):  
    def setUp(self):  
        self.client = app.test_client()  
  
    def test_ping(self):  
        response = self.client.get('/ping')  
        self.assertEqual(response.status_code, 200)  
        self.assertIn('pong', response.data)
```


Test con Postman

Existe la posibilidad de hacer **test automatizados** con la herramienta postman (también con insomnia).

Estos tests pueden correrse de forma individual o en conjunto para validar que nuestra api funcione correctamente.

También se pueden setear variables de entorno para hacer request con información predefinida.

```
pm.test("Status code is 200", () => {  
  pm.response.to.have.status(200);  
});
```

Debugging



Un bug es un error de código en un programa informático

Podemos decir que un bug es cualquier comportamiento error o fallo de un programa.

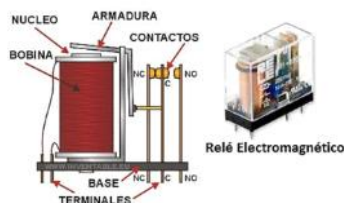
Podemos dividir los bugs en categorías:

- De sintaxis

- De ejecución

- De diseño

Origen del término



Depurar o De buggear

Es el proceso que consiste en la detección y corrección de bugs.

Es un proceso más que habitual en el ambiente del desarrollo tanto en la etapa de creación como en la de mantenimiento de un programa.

Debugger o Depurador

Es una herramienta que se utiliza para el proceso de debugging

Existen muchos debuggers pero casi todos ofrecen las mismas posibilidades.

Uno de los debuggers más famosos es GDB, otro es el debugger de JavaScript de Node.js.

Un debugger puede detener la ejecución del programa e irlo ejecutando línea a línea.
Hasta puede ejecutarlo instrucción a instrucción.

Brinda información sobre variables, scopes, threads, stack trace y de casi todo lo que compone al programa en sí.

El "debugger" más primitivo

Una forma instintiva de hacer debugging es usar "prints" en partes específicas del código. Usar prints como último recurso.

Depurando JavaScript

Iniciando el debugger en VSCode

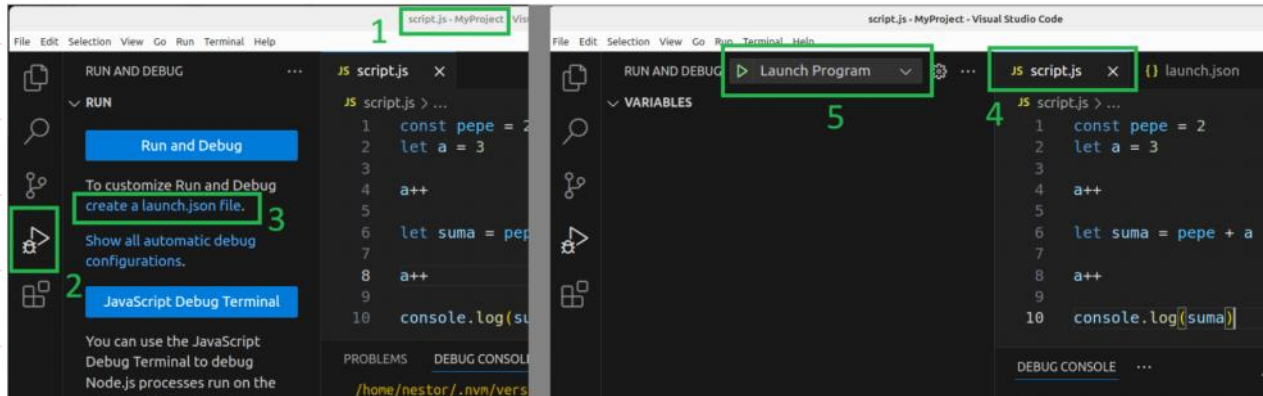
Paso 0: Instalar Node.js

Paso 1: Abrir la carpeta del proyecto en VSCode que tenga el archivo .js a depurar

Paso 2: Ir a la pestaña **run and debug** (el play con el bichito)

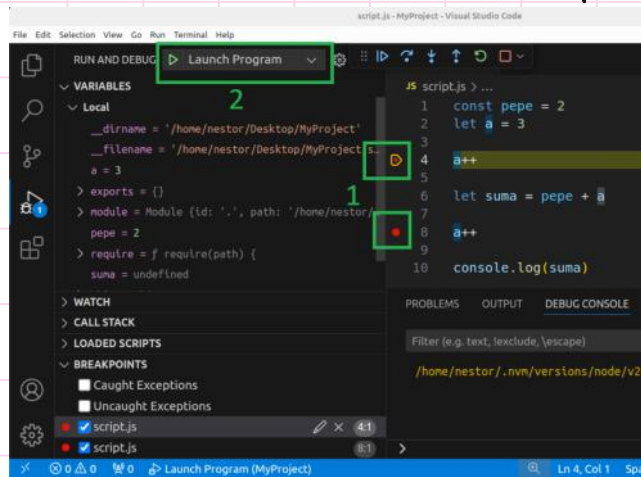
Paso 3: hacer click donde dice "Crear un archivo launch.json" y elegir en el menú la opción "Node.js"

Paso 4: Hacer click arriba a la izquierda en **Launch program**



Breakpoint

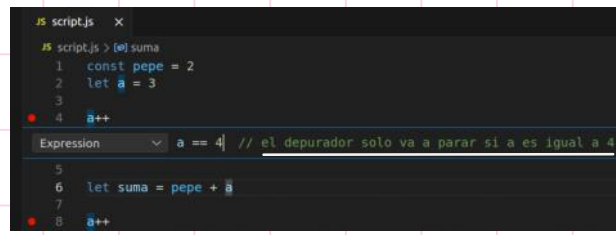
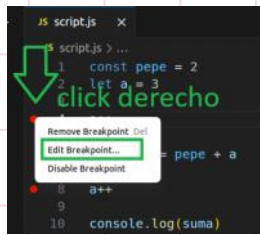
Un breakpoint o punto de parada sirve para que al debuggear el debugger detenga la ejecución en la línea donde colocamos el breakpoint.



Breakpoint Condicional

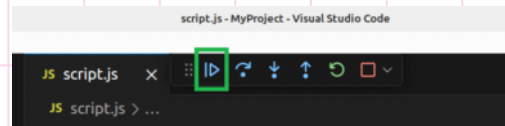
Es un breakpoint que tiene asociada una condición.

El programa se detendrá en esa línea si y sólo si se cumple la condición asociada

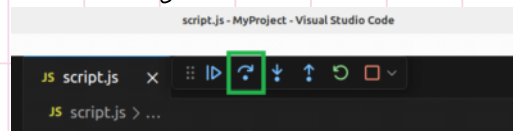


Comandos:

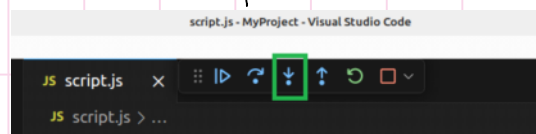
Continue Permite continuar con la ejecución hasta el siguiente breakpoint o hasta el fin del programa.



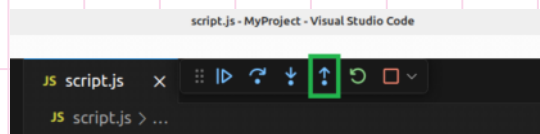
Next Permite avanzar a la línea inmediatamente siguiente o si existe algún breakpoint intermedio, se detiene en él



Step Into Permite ingresar dentro del código de una función para depurar allí. Si en el flujo de ejecución existe un breakpoint se detendrá en él



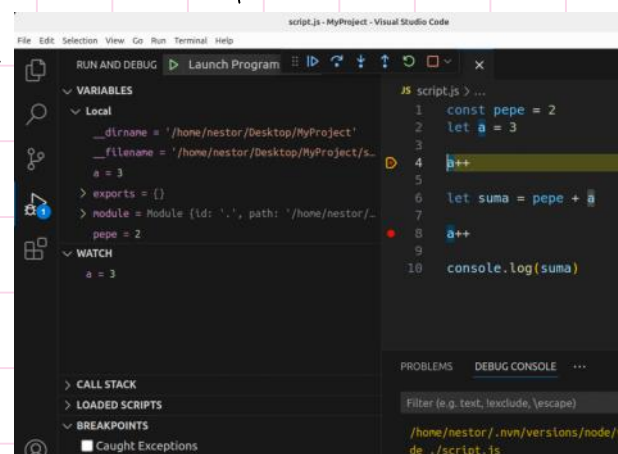
Step Out Sale de la función actual hacia la función que la llamó. Si en el flujo de ejecución existe un breakpoint se detendrá en él



Información de variables

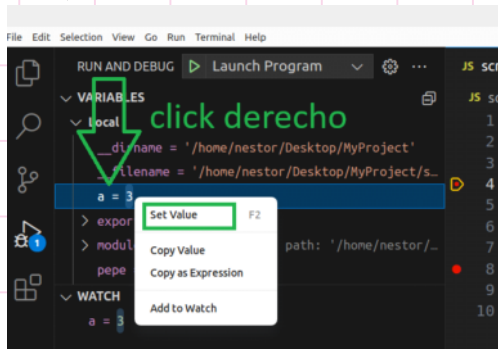
Permite observar el contenido en ese instante de ejecución de las variables

También se le puede dar "seguimiento" (watch) a una variable y cada vez que es usada podemos hacer que el programa se detenga.



Seteo de variables

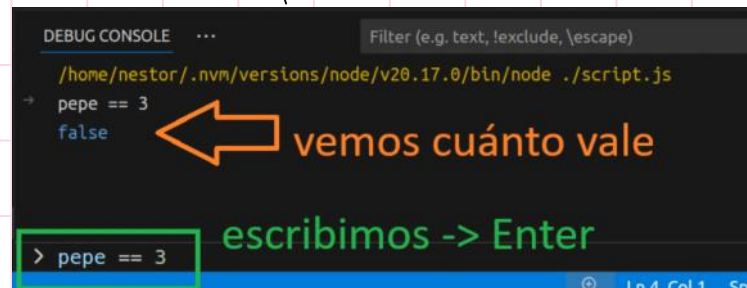
Permite cambiar el valor de las variables en tiempo de ejecución pudiendo así llegar a casos borde o permitiendo hacer análisis del comportamiento del programa en ese contexto.



Evaluar expresiones

Podemos evaluar expresiones con el contenido de las variables en ese instante de ejecución o cualquier otra expresión que nos resulte útil.

Importante: sólo podemos evaluar los valores de las variables que se encuentren disponibles en el scope actual.



Otros debuggers

🌸 Python:

- PDB

🌸 JetBrains:

- IntelliJ IDEA

🌸 Consola de desarrollador de Google Chrome

Un proyecto puede ser visto como una serie de tareas que tienen:

- ⊗ Un objetivo específico de desarrollar un producto o servicio único.
a completarse dentro de ciertas especificaciones.
- ⊗ Fechas de comienzo y fin definidas
- ⊗ Límites presupuestarios.
- ⊗ Recursos que consume.

Complejidad de los proyectos

Tamaño Medido en cantidad de partes o cosas que intervienen y se relacionan

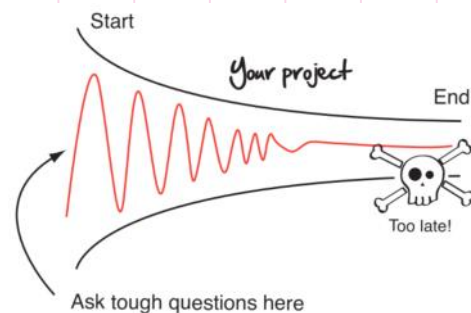
Dificultad Cantidad importante de técnicas y herramientas necesarias para dominar el tema

Variedad Múltiples opciones para solucionar el mismo problema

Cambio Variación de los requerimientos originales

Algunos problemas en la gestión de proyectos

- ⊗ Requerimientos fuera de control
- ⊗ No cumplimiento de los tiempos
- ⊗ Planificados (desvíos)
- ⊗ Estimaciones deficientes
- ⊗ Re-trabajo excesivo
- ⊗ Mala Calidad
- ⊗ Costos excedidos
- ⊗ Insatisfacción del cliente
- ⊗ Insatisfacción de los profesionales participantes



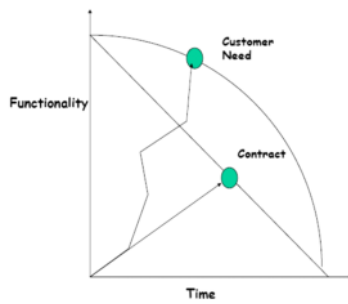
Qué variables tenemos en un proyecto

- Calendario
- Esfuerzo
- Alcance

Project scope



¿Qué quieren los clientes?



YOUR CUSTOMER DISCOVERS WHAT THEY REALLY WANT ...



Entonces... el mercado nos demanda.

BAJO COSTO

ALTA CALIDAD

DIFERENCIACIÓN

REQUIERE: VELOCIDAD + FLEXIBILIDAD

Por qué Scrum

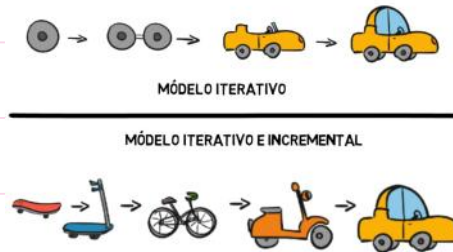
- ❁ Scrum es un proceso ágil que nos permite centrarnos en ofrecer el más alto valor de negocio en el menor tiempo.
- ❁ Nos permite rápidamente y en repetidas ocasiones inspeccionar el producto / servicio real de trabajo (cada dos semanas o un mes).
- ❁ El negocio fija las prioridades. Los equipos se auto-organizan a fin de determinar la mejor manera de entregar las funcionalidades de más alta prioridad.
- ❁ Cada dos semanas o un mes, cualquiera puede ver lo que generamos funcionando y decidir si liberarlo o seguir mejorándolo en otro sprint.
- ❁ Proceso simple que requiere mucha disciplina para que resulte exitoso.
- ❁ Ampliamente utilizado en compañías de todo tipo.

Características

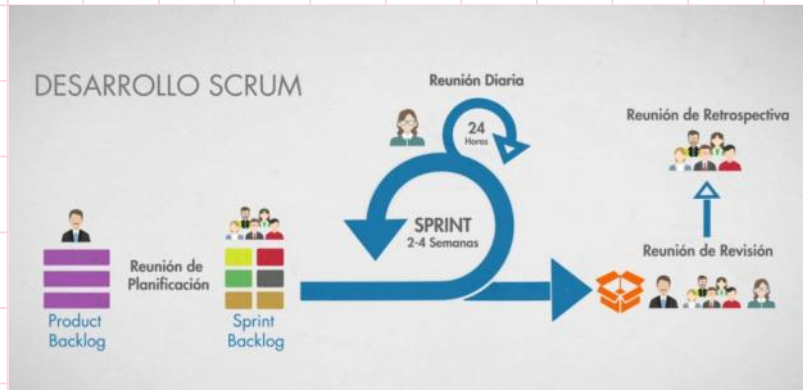
- ❁ Equipos auto-organizados
- ❁ El producto avanza en una serie de "Sprints" de dos semanas a un mes de duración
- ❁ Los requisitos son capturados como elementos de una lista de "Product Backlog"
- ❁ No hay prácticas de Ingeniería Prescritas
- ❁ Utiliza normas generativas para crear un entorno ágil para la entrega de proyectos
- ❁ Uno de los "procesos ágiles".

⊛ Uno de los "procesos ágiles".

Desarrollo iterativo e incremental

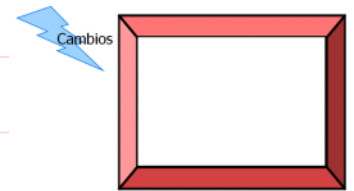


Scrum



No hay cambios en un sprint

La duración de un sprint se planea en torno a cuánto tiempo usted puede comprometerse a mantener los cambios fuera del sprint.



Sólo es posible cambiar el curso de un sprint, abortándolo y sólo lo puede hacer el líder del proyecto si decide que no es viable por alguna de las razones siguientes:

- La tecnología acordada no funciona
- Las circunstancias del negocio han cambiado
- El equipo ha tenido interferencias.

Scrum Framework

Roles

- Product owner
- Scrum Master
- Team

Reuniones

- Sprint planning
- Sprint review
- Sprint retrospective
- Daily scrum meeting

Artefactos

- Product Backlog
- Sprint Backlog
- Burndown charts

Product Backlog

Lista de todo lo necesario para concluir el producto.

- Funcionalidad (user stories): "como usuario quiero poder buscar y reemplazar para evitar tareas repetitivas".
- Tareas pendientes: "mejorar el manejo de excepciones"

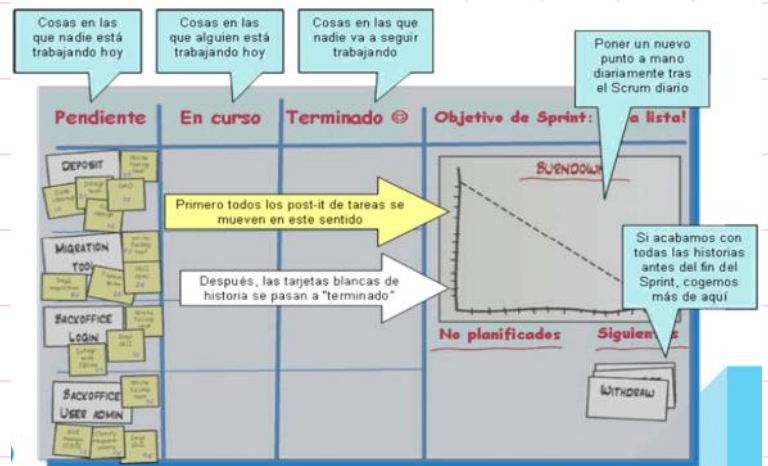
La lista es priorizada por el product owner

- Product manager, marketing, cliente interno, ...

Ejemplo de Product Backlog

	Item #	Description	Est	By
Very High				
	1	Finish database versioning	16	KH
	2	Get rid of unneeded shared Java in database	8	KH
	3	Add licensing	-	-
	3	Concurrent user licensing	16	TG
	4	Demo / Eval licensing	16	TG
	Analysis Manager			
	5	File formats we support are out of date	160	TG
	6	Round-trip Analyses	250	MC
High				
	-	Enforce unique names	-	-
	7	In main application	24	KH
	8	In import	24	AM
	-	Admin Program	-	-
	9	Delete users	4	JM
	-	Analysis Manager	-	-
	-	When items are removed from an analysis, they should show up again in the pick list in lower 1/2 of the analysis tab	-	-
	10	Query	8	TG
	11	Support for wildcards when searching	16	T&A
	12	Sorting of number attributes to handle negative numbers	16	T&A
	13	Horizontal scrolling	12	T&A
	-	Population Genetics	-	-
	14	Frequency Manager	400	T&M
	15	Query Tool	400	T&M
	16	Additional Editors (which ones)	240	T&M
	17	Study Variable Manager	240	T&M
	18	Haplotypes	320	T&M
	19	Add icons for v1.1 or 2.0	-	-
	-	Pedigree Manager	-	-
	20	Validate Derived kindred	4	KH
Medium				
	-	Explorer	-	-
	-	Launch tab synchronization (only show queries/analyses for logged in users)	8	T&A
	21	Delete settings (?)	4	T&A

O mejor aún ... con un tablero kanban



Roles: Product owner

- ⊛ Define las funcionalidades del producto
- ⊛ Es responsable por la rentabilidad del producto (ROI)
- ⊛ Prioriza funcionalidades de acuerdo al valor del mercado/negocio
- ⊛ Ajusta funcionalidades y prioridades en cada iteración si es necesario
- ⊛ Acepta o rechaza los resultados del trabajo del equipo.

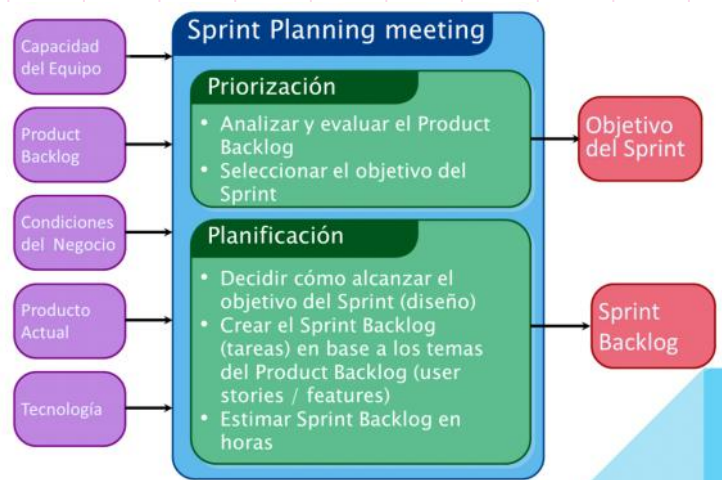
El scrum Master

- ⊗ Representa a la gestión del proyecto
- ⊗ Responsable de promover los valores y prácticas de Scrum
- ⊗ Remueve impedimentos
- ⊗ Se asegura de que el equipo es completamente funcional y productivo
- ⊗ Permite la estrecha cooperación en todos los roles y funciones
- ⊗ Escudo del equipo de interferencias externas.

El team típicamente de 5 a 9 personas.

- ⊗ Multifuncional (Programadores, testers, analistas, diseñadores, etc.)
- ⊗ Los miembros deben ser full-time (puede haber excepciones) ej: Infraestructura, SCM, etc)
- ⊗ Los equipos son auto-organizativos (idealmente, no existen títulos pero a veces se utilizan de acuerdo a la organización.
- ⊗ Sólo puede haber cambio de miembros entre los sprints.

Reuniones: Planning Meeting



Planificación del Sprint

- ⊗ El equipo selecciona los temas a partir del Product Backlog que pueden comprometerse a completar.
- ⊗ Se crea el Sprint Backlog
- ⊗ Se identifican tareas y cada una es estimada (1-16hs)
- ⊗ Realizado colaborativamente, no solo por el scrum Master

Daily Scrum

Parámetros

- ⊗ Diaria
- ⊗ Dura 15 minutos
- ⊗ Parados

No para la solución del problema

- ⊗ Todo el mundo está invitado
- ⊗ Sólo los miembros del equipo, Scrum Master y Product Owner pueden hablar
- ⊗ Ayuda a evitar otras reuniones innecesarias
- ⊗ No se permite entrar en divagaciones o salirse del guión

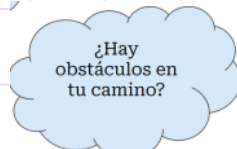
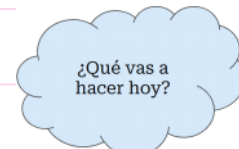
Todos los días en el mismo sitio y a la misma hora

Cuando un miembro informa algo de interés para otros, o necesita ayuda de otros, estos se reúnen al terminar la revisión diaria.

Todos responden 3 Preguntas

No es dar un status report al Scrum Master

Se trata de compromisos delante de pares.



Estimación: Unidad de medida

Es necesario establecer una unidad para poder estimar, podrían ser puntos dependiendo del tamaño.

Estimación: Planning Poker

Planning poker es un juego dentro del equipo que nos permite estimar

- ⊗ Se tienen tarjetas con números que representan puntos dependiendo del tamaño de la tarea.
- ⊗ Cada participante tiene una tarjeta donde coloca lo que cree que esa tarea vale
- ⊗ Se muestran todas las tarjetas
- ⊗ Si las estimaciones son similares $\Rightarrow$ se toma esa estimación
- ⊗ Caso contrario se discuten los puntos de vista y se vuelve a estimar



Retrospectiva del Scrum

- ⊛ Acuden el equipo y el Scrum Master, y opcionalmente el propietario del producto
- ⊛ Todos los miembros del equipo responden a dos preguntas:
 - ¿Qué cosas fueron bien en el último sprint?
 - ¿Qué cosas se podrían mejorar?
- ⊛ El Scrum Manager anota todas las respuestas y el equipo prioriza las mejoras posibles
- ⊛ El Scrum Manager no proporciona respuestas, sino que ayuda al equipo a encontrar la mejor forma de trabajar con Scrum
- ⊛ Las acciones de mejora localizadas que se puedan implementar en el próximo Sprint deben introducirse en la Pila de Producto como elementos no funcionales.

Kanban



Kanban es

- ⊛ Dividir el trabajo en partes, escribir cada elemento en una tarjeta y ponerla en la pared
- ⊛ Utilice columnas nombradas para ilustrar que cada elemento está en el flujo de trabajo.
- ⊛ Limit work in progress (WIP), asignar límites explícitos sobre cómo muchas historias podrían estar en curso en cada estado del flujo de trabajo

Medir el leadtime (tiempo promedio para completar una tarea, conocido como "tiempo de ciclo"), optimizar el proceso de forma de hacer el tiempo desperdiciado tan pequeño y predecible posible.

